Course Discussions

Solution: Number factors

Problem Statement

Given a number 'n', implement a method to count how many possible ways there are to express 'n' as the sum of 1, 3, or 4.

Example 1:

```
n : 4
Number of ways = 4
Explanation: Following are the four ways we can express 'n' : {1,1,1,1},
{1,3}, {3,1}, {4}
```

Example 2:

```
n : 5
Number of ways = 6
Explanation: Following are the six ways we can express 'n' :
{1,1,1,1,1}, {1,1,3}, {1,3,1}, {3,1,1},
{1,4}, {4,1}
```

Let's first start with a recursive brute-force solution.

Basic Solution

For every number 'i', we have three option: subtract either 1, 3, or 4 from 'i' and recursively process the remaining number. So our algorithm will look like:

Java

```java
class Solution {

  public int countWays(int n) {
    if( n == 0)
      return 1; // base case, we don't need to subtract any thing, so there is only one

    if(n == 1)
      return 1; // we can subtract 1 to be left with zero, and that is the only way

    if(n == 2)
      return 1; // we can subtract 1 twice to get zero and that is the only way

    if(n == 3)
      return 2; // '3' can be expressed as {1,1,1}, {3}

    // if we subtract 1, we are left with 'n-1'
    int subtract1 = countWays(n-1);
    // if we subtract 3, we are left with 'n-3'
    int subtract3 = countWays(n-3);
    // if we subtract 4, we are left with 'n-4'
    int subtract4 = countWays(n-4);
```

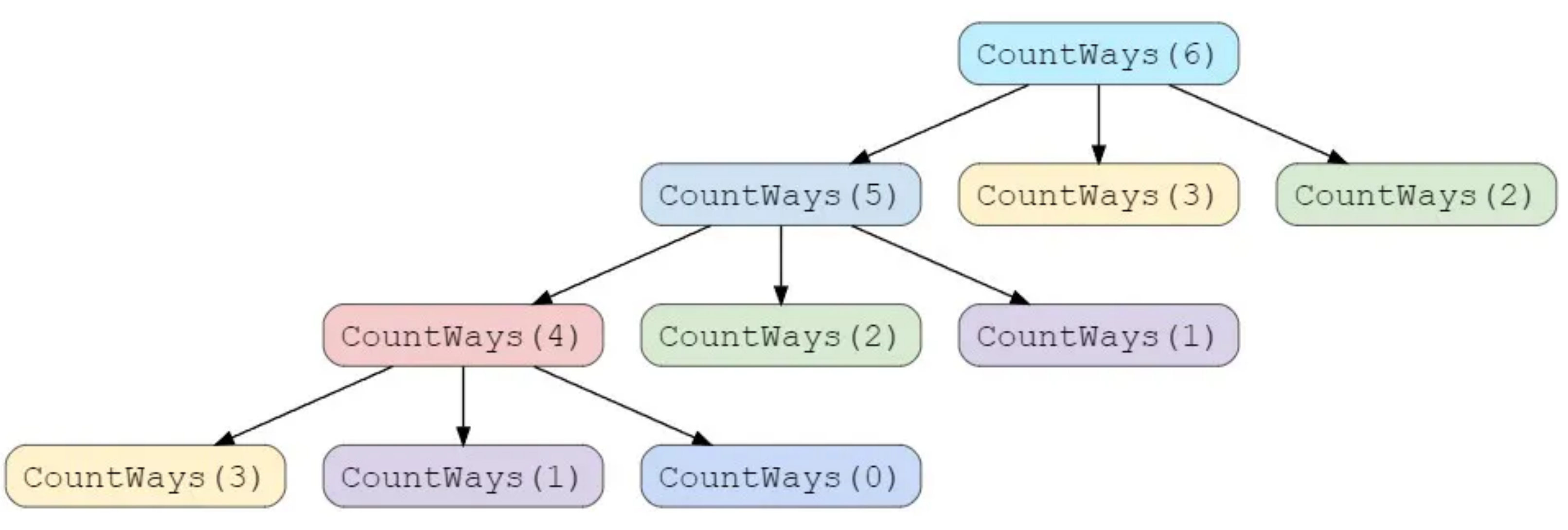
Run Code Result Stdout

You must run your code first

Execute

The time complexity of the above algorithm is exponential $O(3^n)$. The space complexity is $O(n)$ which is used to store the recursion stack.

Let's visually draw the recursion for `CountWays(5)` to see the overlapping subproblems:



Recursion tree for calculating Fibonacci numbers

We can clearly see the overlapping subproblem pattern: `CountWays(3)`, `CountWays(2)` and `CountWays(1)` have been called twice. We can optimize this using memoization to store the results for subproblems.

Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems. Here is the code:

Java

```java
class Solution {

  public int countWays(int n) {
    int dp[] = new int[n + 1];
    return countWaysRecursive(dp, n);
  }

  public int countWaysRecursive(int[] dp, int n) {
    if (n == 0)
      return 1; // base case, we don't need to subtract any thing, so there is only one

    if (n == 1)
      return 1; // we can take subtract 1 to be left with zero, and that is the only way

    if (n == 2)
      return 1; // we can subtract 1 twice to get zero and that is the only way

    if (n == 3)
      return 2; // '3' can be expressed as {1,1,1}, {3}

    if (dp[n] == 0) {
```

Run Code Result Stdout

You must run your code first

Execute

Bottom-up Dynamic Programming

Let's try to populate our `dp[]` array from the above solution, working in a bottom-up fashion. As we saw in the above code, every `CountWaysRecursive(n)` is the sum of the three counts. We can use this fact to populate our array.

Code

Here is the code for our bottom-up dynamic programming approach:

Java

```java
class Solution {

  public int countWays(int n) {
    // bottom up
    int n0 = 1, n1 = 1, n2 = 1, n3 = 2;
    for(int i = 4; i <= n; i++){
      int temp = n0 + n1 + n3;
      n0 = n1;
      n1 = n2;
      n2 = n3;
      n3 = temp;
    }
    return n3;
  }

  public static void main(String[] args) {
    Solution en = new Solution();
    System.out.println(en.countWays(4));
    System.out.println(en.countWays(5));
    System.out.println(en.countWays(6));
  }
```

Run Code Result Stdout

You must run your code first

Execute

The above solution has a time and space complexity of $O(n)$ and $O(1)$ respectively.

Fibonacci number pattern

We can clearly see that this problem follows the Fibonacci number pattern. However, every number in a Fibonacci series is the sum of the previous two numbers, whereas in this problem every count is a sum of previous three numbers: previous-1, previous-3, and previous-4. Here is the recursive formula for this problem:

```
CountWays(n) = CountWays(n-1) + CountWays(n-3) + CountWays(n-4), for n ≥ 4
```

← Previous
Number Factors

Next →
Minimum Jumps To Reach The End

☐ Mark as Completed